

Express.js Master Class:

Comprehensive Backend Development Guide

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. This guide covers everything required to master Express.js for production-ready backend applications.

1. Setting Up the Environment

To initialize a new Express.js application, execute the following commands in your terminal:

```
mkdir express-backend  
cd express-backend  
npm init -y  
npm install express
```

Create an index.js file and add the basic boilerplate to start a server:

```
const express = require('express');  
const app = express();  
const PORT = process.env.PORT || 3000;  
  
app.get('/', (req, res) => {  
  res.send('Hello, Express World!');  
});  
  
app.listen(PORT, () => {  
  console.log(`Server is running on port ${PORT}`);  
});
```

2. Routing System

Routing determines how an application responds to a client request to a specific endpoint.

Express supports standard HTTP methods: GET, POST, PUT, and DELETE.

Route Parameters and Query Strings

Route parameters are named URL segments used to capture values at specific positions. Query strings are key-value pairs following a question mark in the URL.

```
// Handle Route Parameters (e.g., /users/42)
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`User ID requested: ${userId}`);
});

// Handle Query Strings (e.g., /search?q=javascript)
app.get('/search', (req, res) => {
  const searchQuery = req.query.q;
  res.send(`Search query: ${searchQuery}`);
});
```

3. Middleware Architecture

Middleware functions are functions that have access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle.

Types of Middleware

- **Application-level middleware:** Bound to an instance of the app object using `app.use()`.
- **Router-level middleware:** Bound to an instance of `express.Router()`.
- **Built-in middleware:** Provided by Express, such as `express.json()` and `express.static()`.
- **Error-handling middleware:** Defined with four arguments instead of three: (err, req, res, next).

Custom Middleware Example

```
const requestLogger = (req, res, next) => {
  console.log(`${req.method} request made to ${req.url} at ${new Date().toISOString()}`);
  next(); // Pass control to the next middleware function
};
```

```
app.use(requestLogger);
app.use(express.json()); // Built-in middleware to parse JSON bodies
```

4. Global Error Handling

Proper error handling ensures robustness and prevents server crashes from unexpected runtime exceptions.

```
// Global error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    message: err.message || 'Internal Server Error'
  });
});
```

5. Recommended Project Directory Structure

For scalable applications, organizing code using a clean Model-View-Controller (MVC) or layered architecture is highly recommended.

Directory / File	Purpose
/config	Database configuration and environment variables setup.
/controllers	Functions that handle application logic and interact with models.
/middleware	Custom authorization, logging, and input validation functions.

Directory / File	Purpose
/models	Database schemas and data models (e.g., Mongoose, Sequelize).
/routes	Route definitions routing to respective controller actions.
index.js	Entry point of the Express application.